

Beautifying C++ Console Output

<https://gemini.google.com/app/b8378f1b13f62897>

User prompt: namespace Color { const std::string RESET = "\033[0m"; const std::string BOLD = "\033[1m"; const std::string DIM = "\033[2m"; const std::string CYAN = "\033[36m"; const std::string GREEN = "\033[32m"; const std::string YELLOW = "\033[33m"; const std::string BLUE = "\033[34m"; const std::string MAGENTA = "\033[35m"; const std::string GRAY = "\033[90m"; const std::string RED = "\033[31m"; // Status Badges const std::string CLIENT_BADGE = "\033[1;37;44m CLIENT \033[0m"; // White on Blue const std::string SERVER_BADGE = "\033[1;37;42m SERVER \033[0m"; // White on Green const std::string MATCH_BADGE = "\033[1;30;42m MATCH \033[0m"; // Black on Green } For all the cout I give you, you have to give me a beautified cout as the current is not that good. std::cout << "[?] Unknown " << FingerprintDatabase::kind_name(kind) << " fingerprint " << hash << ". Enter verified client/server name " << " (empty to skip): " << std::flush; std::cout << " | [V-Version] " << to_hex(ch.client_version) << "\n" << " | [V-SNI] " << (ch.has_sni ? ch.sni : "<none>") << "\n" << " | [V-ALPN] " << (ch.first_alpn.empty() ? "<none>" : ch.first_alpn) << "\n" << " | [V-Ciphers] Count: " << ch.cipher_suites.size() << " ["; std::cout << to_hex(ch.cipher_suites[i]); std::cout << to_hex(ch.extensions[i]); std::cout << to_hex(ch.supported_groups[i]); std::cout << to_hex(ch.signature_algorithms[i]); std::cout << "\n" << " | [V-PreHash-JA3] " << ja3.raw_string << "\n" << " | [V-PreHash-JA4b] " << ja4.raw_ja4_b << "\n" << " | [V-PreHash-JA4c] " << ja4.raw_ja4_c << "\n"; std::cout << " | [V-WireVersion] " << to_hex(sh.server_version) << "\n" << " | [V-Negotiated] " << to_hex(sh.selected_version) << "\n" << " | [V-Cipher] " << to_hex(sh.selected_cipher) << "\n" << " | [V-ALPN] " << (sh.first_alpn.empty() ? "<none>" : sh.first_alpn) << "\n" << " | [V-Extensions] Count: " << sh.extensions.size() << " ["; std::cout << to_hex(sh.extensions[i]); std::cout << "\n" << " | [V-PreHash-JA3S] " << ja3s.raw_string << "\n" << " | [V-PreHash-JA4S] " << ja4s.raw_ja4_c << "\n";

Response: To make these console outputs look clean, readable, and modern, apply your existing Color constants consistently. By leveraging tree-like ASCII characters, soft gray accents for metadata labels, and bold colors for dynamic values, the parsed TLS parameters become much easier to scan.

Refactored C++ Output Code

```
// -----
// 1. Interactive Prompt
// -----
std::cout << Color::YELLOW << Color::BOLD << "[?]" << Color::RESET
          << " Unknown " << Color::CYAN << FingerprintDatabase::kind_name(kind) << Color::RESET
          << " fingerprint " << Color::MAGENTA << hash << Color::RESET << ".\n"
          << Color::DIM << " | Enter verified name (or press Enter to skip): " << Color::RESET
          << std::flush;

// -----
// 2. Client Hello (CH) Inspection
// -----
std::cout << Color::GRAY << " | " << Color::RESET << Color::BOLD << "Version" << Color::RESET << C
          << Color::GRAY << " | " << Color::RESET << Color::BOLD << "SNI" << Color::RESET << (
          << Color::GRAY << " | " << Color::RESET << Color::BOLD << "ALPN" << Color::RESET << (
          << Color::GRAY << " | " << Color::RESET << Color::BOLD << "Ciphers" << Color::RESET << C

// --- Loop: Ciphers ---
for (size_t i = 0; i < ch.cipher_suites.size(); ++i) {
    std::cout << Color::YELLOW << to_hex(ch.cipher_suites[i]) << Color::RESET
              << (i + 1 < ch.cipher_suites.size() ? Color::GRAY + ", " + Color::RESET : "");
}

// --- Loop: Extensions ---
// (Print opening bracket prior to loop)
for (size_t i = 0; i < ch.extensions.size(); ++i) {
    std::cout << Color::YELLOW << to_hex(ch.extensions[i]) << Color::RESET
              << (i + 1 < ch.extensions.size() ? Color::GRAY + ", " + Color::RESET : "");
}

// --- Loop: Supported Groups ---
// (Print opening bracket prior to loop)
for (size_t i = 0; i < ch.supported_groups.size(); ++i) {
    std::cout << Color::YELLOW << to_hex(ch.supported_groups[i]) << Color::RESET
              << (i + 1 < ch.supported_groups.size() ? Color::GRAY + ", " + Color::RESET : "");
}

// --- Loop: Signature Algorithms ---
// (Print opening bracket prior to loop)
for (size_t i = 0; i < ch.signature_algorithms.size(); ++i) {
    std::cout << Color::YELLOW << to_hex(ch.signature_algorithms[i]) << Color::RESET
              << (i + 1 < ch.signature_algorithms.size() ? Color::GRAY + ", " + Color::RESET : "");
}

std::cout << Color::GRAY << " ]" << Color::RESET << "\n"
          << Color::GRAY << " | " << Color::RESET << Color::BOLD << "JA3 String" << Color::RESET << C
          << Color::GRAY << " | " << Color::RESET << Color::BOLD << "JA4b" << Color::RESET << C
          << Color::GRAY << " | " << Color::RESET << Color::BOLD << "JA4c" << Color::RESET << C

// -----
// 3. Server Hello (SH) Inspection
// -----
std::cout << Color::GRAY << " | " << Color::RESET << Color::BOLD << "Wire Ver" << Color::RESET << C
          << Color::GRAY << " | " << Color::RESET << Color::BOLD << "Negotiated" << Color::RESET << C
```

```

    << Color::GRAY << "      └─ " << Color::RESET << Color::BOLD << "Cipher          " << Color::RESET << C
    << Color::GRAY << "      └─ " << Color::RESET << Color::BOLD << "ALPN          " << Color::RESET << (
    << Color::GRAY << "      └─ " << Color::RESET << Color::BOLD << "Extensions    " << Color::RESET << C

// --- Loop: Server Extensions ---
for (size_t i = 0; i < sh.extensions.size(); ++i) {
    std::cout << Color::YELLOW << to_hex(sh.extensions[i]) << Color::RESET
               << (i + 1 < sh.extensions.size() ? Color::GRAY + ", " + Color::RESET : "");
}

std::cout << Color::GRAY << " ]" << Color::RESET << "\n"
          << Color::GRAY << "      └─ " << Color::RESET << Color::BOLD << "JA3S String " << Color::RESET << C
          << Color::GRAY << "      └─ " << Color::RESET << Color::BOLD << "JA4S          " << Color::RESET << C

```

Key Enhancements

- **Tree Hierarchy** (└─, └─): Replaced the dense | [V-...] syntax with lightweight tree connectors.
 - **Separation of Concerns**: Hex values use **Yellow/Cyan**, string fields (SNI, ALPN) use **Green**, and raw pre-hash strings use **Dim** styling to avoid overpowering key metadata.
 - **Formatted Arrays**: Loop elements now feature comma separators with muted gray brackets ([...]).
 - **Clean Padding**: Field titles (Version, Negotiated, Ciphers) are aligned using standard string padding instead of heavy square-bracket tags like [V-PreHash-JA4c].
-